

 INSTITUTO FEDERAL de Goiás CAMPUS Anápolis	INSTITUTO FEDERAL DE GOIÁS
	Campus Anápolis
	Disciplina: Programação Web
	Professor(a): Luiz Fernando Batista Loja
	Trabalho: Aplicação Web de E-commerce com Sinatra e ActiveRecord

☰ TRABALHO

Aplicação Web de E-commerce com Sinatra e ActiveRecord

👤 Grupo

Até 2
alunos

📅 Entrega

30 de junho de
2026, até as 23h59

📁 Forma de entrega

Arquivo .zip entregue
presencialmente em sala
de aula

📋 Descrição

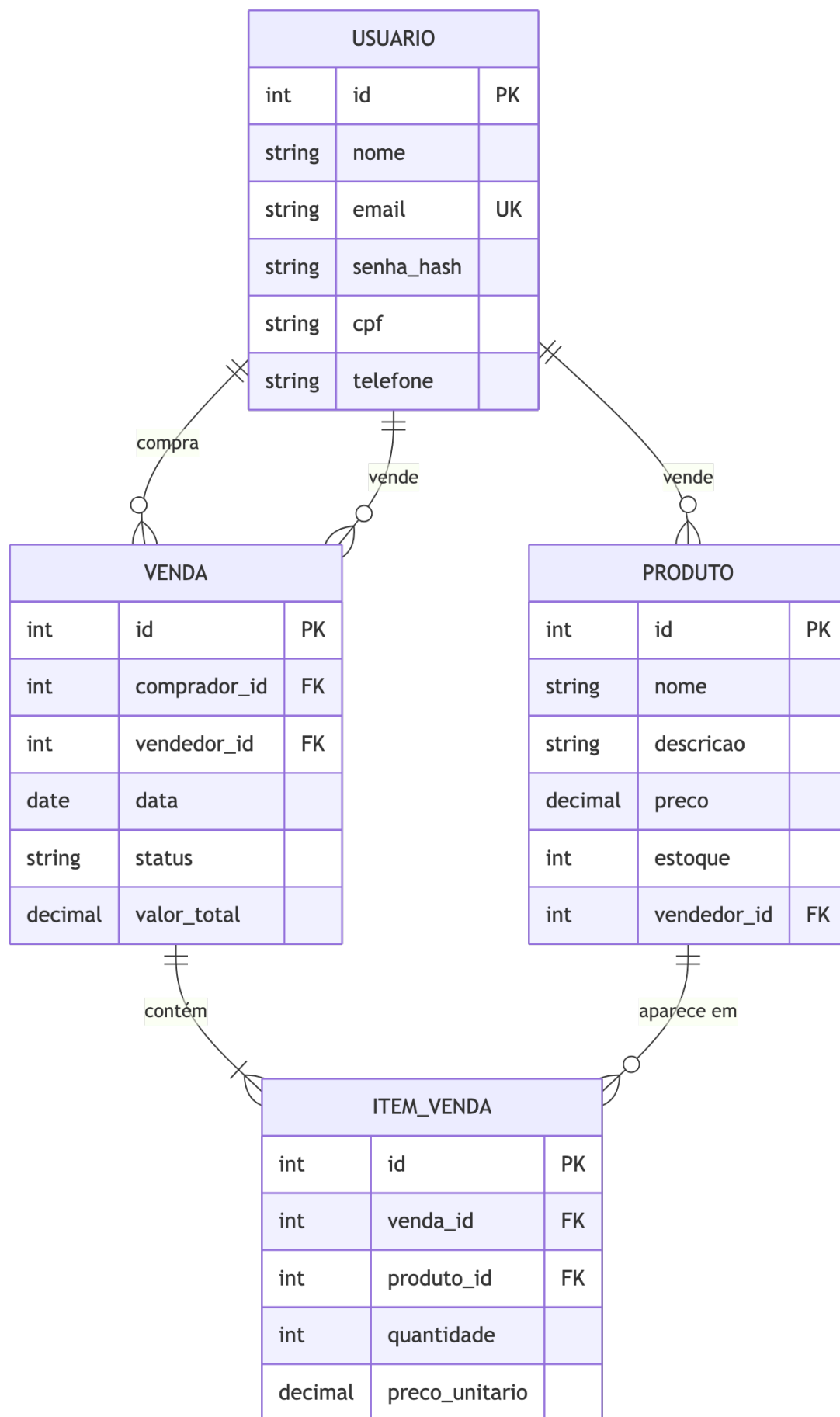
Desenvolver uma aplicação web em **Ruby/Sinatra** integrada ao **ActiveRecord** que implemente um sistema de **compra e venda de produtos** (e-commerce simplificado). Um mesmo **usuário** pode atuar como **vendedor** (cadastrando produtos próprios) e/ou como **comprador** (efetuando compras) — o papel é inferido pelas associações.

A aplicação deve expor as rotas HTTP necessárias, usar **templates ERB** para renderizar as páginas, conter **formulários HTML** para entrada de dados e persistir tudo em um banco **SQLite** via **migrations**. O domínio é fixo (e-commerce) e o modelo de dados está especificado nas figuras a seguir.

Modelo de dados (MER)

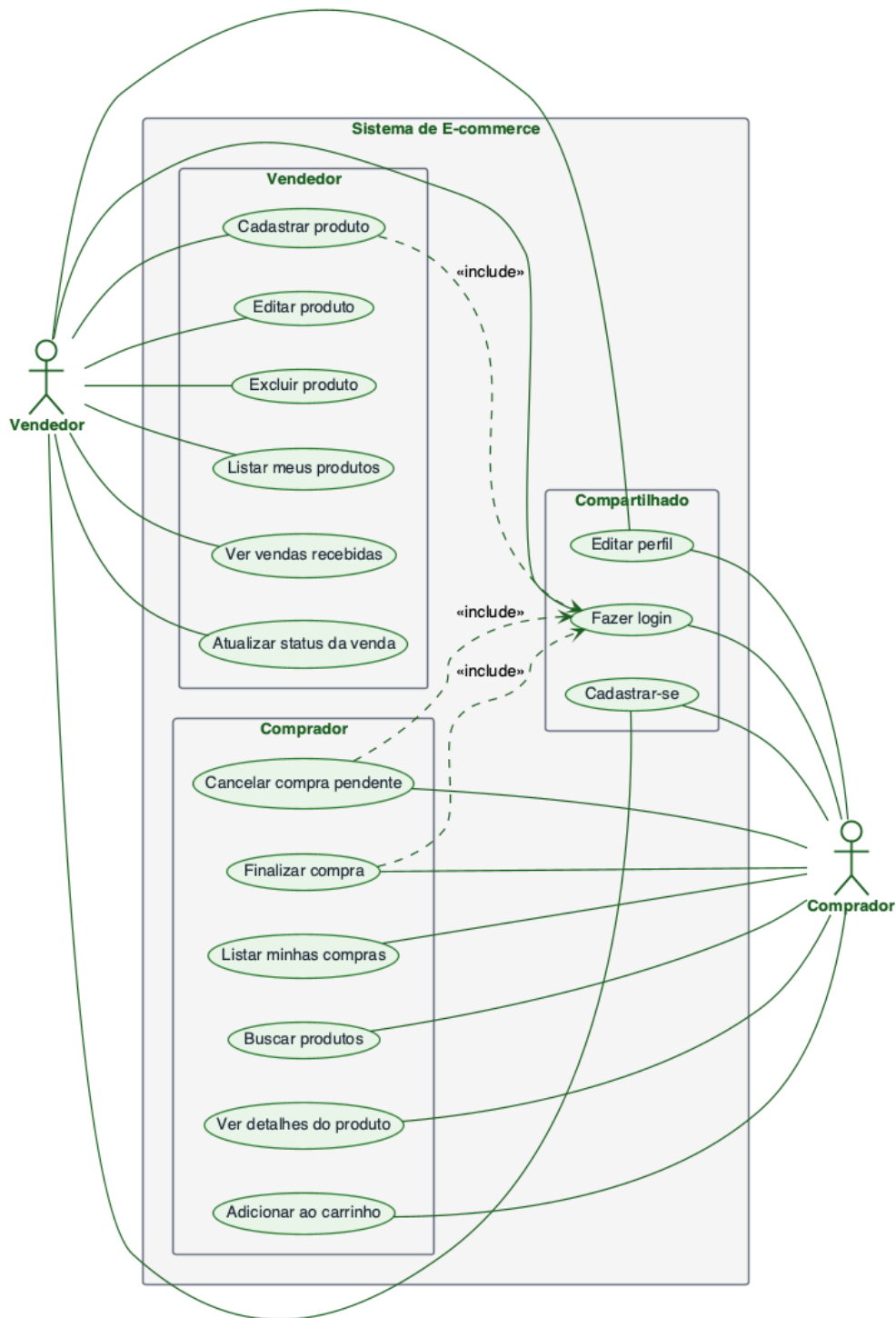
Estrutura mínima exigida — quatro entidades e seus relacionamentos.

Modelo Entidade-Relacionamento – E-commerce



Casos de uso

Funcionalidades por ator. O mesmo usuário pode assumir os dois papéis.



✓ Requisitos

Tecnologias obrigatórias:

- Aplicação web escrita em Ruby, servida pelo Sinatra, com persistência via ActiveRecord em banco SQLite.
- Páginas renderizadas com templates ERB, formulários HTML e um layout comparti-

lhado com cabeçalho e rodapé.

- Mensagens de feedback de sucesso e de erro exibidas ao usuário a cada ação.
- Documentação do projeto em arquivo README com instruções claras de instalação e execução.

Modelo de dados: o sistema deve possuir as quatro entidades apresentadas no MER acima (Usuário, Produto, Venda e Item de Venda), com os atributos especificados em cada uma e as chaves estrangeiras indicadas. Não é permitido reduzir o número de entidades; é permitido acrescentar atributos opcionais quando fizer sentido para o domínio.

Regras de relacionamento entre as entidades:

- Um mesmo usuário pode atuar como vendedor (cadastrando produtos próprios) e como comprador (efetuando compras). O papel é inferido pelos relacionamentos: quem cadastra produtos é vendedor; quem efetua compras é comprador.
- Cada produto pertence a um único vendedor, e um vendedor pode ter vários produtos.
- Cada venda pertence a um comprador e a um vendedor distintos. Um usuário pode ter várias vendas como vendedor e várias compras como comprador.
- Uma venda é composta por um ou mais itens, e cada item refere-se a um produto, com sua quantidade e o preço unitário praticado no momento da compra.

Regras de negócio e validações mínimas:

- O e-mail do usuário é obrigatório, deve ter formato válido e não pode se repetir entre usuários.
- O CPF do usuário é obrigatório.
- O nome do produto é obrigatório; o preço deve ser positivo e o estoque não pode ser negativo.
- A situação da venda deve assumir somente um dos valores: pendente, paga, enviada, entregue ou cancelada.
- A quantidade de cada item de venda deve ser maior que zero.
- Ao finalizar uma compra, o estoque do produto deve ser debitado e o valor total da venda deve ser calculado automaticamente, dentro de uma transação que falhe inteira se algum item não puder ser reservado.

✓ Requisitos

Funcionalidades a implementar: as rotas e telas devem permitir, no mínimo, que cada ator realize as ações apresentadas no diagrama de casos de uso. Em particular:

- **Todo usuário** deve poder se cadastrar, autenticar e editar o próprio perfil.
- **Vendedor** deve poder cadastrar, editar, excluir e listar os próprios produtos; consultar as vendas recebidas; e avançar a situação de cada venda, seguindo a sequência natural até a entrega.

- **Comprador** deve poder buscar produtos, ver detalhes, adicionar itens a um carrinho, finalizar a compra, listar as próprias compras e cancelar uma compra ainda pendente.

Testes automatizados (obrigatório): o grupo deve produzir uma suíte de testes que exercite o sistema em três camadas, separadas em pastas dedicadas dentro do projeto.

- **Camada de modelo** — verifica os atributos, os relacionamentos, as validações de presença, unicidade e formato, e as regras de negócio (por exemplo, o estoque ser debitado quando uma compra é finalizada).
- **Camada de rotas e controladores** — exercita cada rota da aplicação, verificando o código de retorno HTTP, os redirecionamentos esperados e o efeito real no banco de dados.
- **Camada de interface** — simula o fluxo de uso real: um usuário se cadastra, autentica, um vendedor publica um produto, um comprador realiza a compra e o vendedor atualiza a situação até a entrega.

A suíte deve poder ser executada por um único comando, descrito no README.

Entregáveis

1. Arquivo `.zip` contendo o projeto completo, entregue **em sala de aula** na data marcada
2. Código-fonte completo no zip, incluindo `Gemfile.lock`, migrations e a pasta `spec/` com os testes RSpec funcionando
3. Arquivo `README.md` cobrindo: dependências, comandos para rodar migrations, comando para iniciar o servidor, comando para rodar os testes e prints de cada tela (cadastro, login, listagem de produtos, carrinho, finalizar compra, minhas compras, vendas recebidas)
4. **Não incluir** no zip: pasta `.bundle/`, `node_modules/`, banco `*.sqlite3` populado nem a pasta `spec/tmp/`
5. Apresentação de **10 minutos** em sala demonstrando: (a) vendedor cadastra produto; (b) comprador realiza compra; (c) vendedor atualiza status; (d) suíte de testes passando

Critérios de avaliação

- **25%** — Modelo de dados aderente ao MER (4 entidades, FKs, associações AR corretas)
- **25%** — Funcionalidades dos casos de uso implementadas (CRUD vendedor + fluxo de compra + status)
- **20%** — Suíte de testes RSpec cobrindo as 3 camadas (modelo, controlador, interface)
- **10%** — Validações e tratamento de erros (estoque, status, unicidade de email)
- **10%** — Qualidade do código (organização, nomes, indentação, ausência de duplicação)
- **5%** — Qualidade da apresentação e domínio do conteúdo

- 5% — README.md claro e completo

Dica

Sugestão de ordem de ataque: (1) criar migrations e modelos com as 4 tabelas; (2) escrever os testes de modelo (associações + validações) já em paralelo; (3) implementar autenticação simples e CRUD de produto; (4) implementar o fluxo de compra (criar **Venda**, criar **ItemVendas**, debitar **estoque**, calcular **valor_total** dentro de uma transação); (5) telas do comprador e do vendedor; (6) atualização de status. Use `rake db:create` e `rake db:migrate` para preparar o banco. Rode `bundle exec rspec` a cada mudança e mantenha a suíte verde antes de partir para a próxima feature.

Atenção

Domínio fixo. O modelo de dados e os casos de uso são os especificados acima — não há escolha de domínio neste trabalho. Você pode adicionar atributos opcionais (ex: **avaliacao** no **Produto**, **endereco_entrega** na **Venda**) desde que o núcleo do MER seja respeitado.

Plágio será reprovado. Cada grupo deve desenvolver seu próprio código. Trechos copiados sem citação ou trabalhos idênticos entre grupos resultarão em **nota zero** para todos os envolvidos.